

Reviewer Pack — Minimal Lattice Point Counting Complexity and Communication ...

Entry c2d9bc03d031 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 10:03:16 UTC

Minimal Lattice Point Counting Complexity and Communication Complexity Rank Correlation

Entry ID: c2d9bc03d031 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 10:03:16 UTC

1. Conjecture

Field A (mathematical branch): Lattice Theory (specifically, counting lattice points) **Field B** (complexity object): Communication Complexity

Statement:

For every n -ary communication protocol P with rank $r(P)$, the minimal number of lattice points in any d -dimensional integer lattice that is the input domain for P is $\Omega(r(P)^2)$. Furthermore, there exists a polynomial-time constructive mapping from each instance of P to an integer lattice L_P such that the number of lattice points in L_P is $\Theta(r(P)^2)$.

Rationale (proposer's reasoning):

Lattice theory provides a natural framework for studying discrete structures and their properties. Counting lattice points can capture the complexity of computing certain combinatorial problems, which may be related to communication complexity. This conjecture proposes that the rank of a communication protocol, which measures its difficulty, is related to the geometric complexity of its input domain.

Taxonomy category: LATTICE_COUNTING (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 773ef27357be9932

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the number of lattice points and the square of the rank exceeds 0.7, based on at least 30 seeds with a p -value < 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - lattice theory AND counting lattice points AND communication complexity - communication complexity rank AND lattice point count $\Omega(r^2)$ - constructive mapping FROM communication protocol TO integer lattice WITH lattice points $\Theta(r^2)$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_protocol_instances(n):
        protocols = []
        for _ in range(30): # Ensure at least 30 instances per seed
            rank = random.randint(1, 5) # Varying ranks from 1 to 5
            protocol = {
                'rank': rank,
                'input_size': n
            }
            protocols.append(protocol)
        return protocols

    def construct_lattice(protocol):
        rank = protocol['rank']
        input_size = protocol['input_size']
        lattice_points = []
        for i in range(input_size):
            for j in range(rank):
                lattice_points.append((i, j))
        return lattice_points

    def count_lattice_points(lattice_points):
        return len(lattice_points)

    def measure_rank(protocol):
        return protocol['rank']
```

```

n_values = [5, 10, 15, 20, 30, 40]
total_points = 0
total_ranks = 0

for n in n_values:
    protocols = generate_protocol_instances(n)
    for protocol in protocols:
        lattice_points = construct_lattice(protocol)
        points_count = count_lattice_points(lattice_points)
        rank = measure_rank(protocol)
        total_points += points_count
        total_ranks += rank ** 2

if total_ranks == 0:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": 180, # 30 instances per n * 6 values of n
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_points = total_points / 180
mean_ranks_squared = total_ranks / 180

# Calculate Pearson correlation coefficient
numerator = sum((points_count - mean_points) * (rank ** 2 - mean_ranks_squared)
                for points_count, rank in zip([count_lattice_points(construct_lattice(protocol))
                                              for protocol in generate_protocol_instances(40)]
                                              [measure_rank(protocol) for protocol in generate_protocol_instances(40)]))

denominator = math.sqrt(sum((points_count - mean_points) ** 2
                             for points_count in [count_lattice_points(construct_lattice(protocol))
                                                  for protocol in generate_protocol_instances(40)]))
                        * math.sqrt(sum((rank ** 2 - mean_ranks_squared) ** 2
                                         for rank in [measure_rank(protocol) for protocol in generate_protocol_instances(40)])))

if denominator == 0:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": 180, # 30 instances per n * 6 values of n
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

correlation_coefficient = numerator / denominator

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": 180, # 30 instances per n * 6 values of n
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient > 0.7,
    "counterexample": ""
}

```


9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The Pearson correlation coefficient between the number of lattice points and the square of the rank did not exceed 0.7 for at least 30 seeds with a p- | next: Increase the range of n tested to confirm the conjecture for general cases or explore alternative metrics that may be more suitable for this type of analysis.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20159
2	propose	ollama_remote	glm4:latest	0	0	23214
3	propose	ollama_remote	glm4:latest	0	0	24769
4	preregistration	ollama_remote	glm4:latest	0	0	9502
5	novelty	ollama_remote	glm4:latest	0	0	27216
6	novelty	ollama_remote	glm4:latest	0	0	20127
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13140
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20785
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12408
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22759
11	critic	ollama_remote	glm4:latest	0	0	14837
12	judge	ollama_remote	glm4:latest	0	0	9282

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 218196 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/c2d9bc03d031.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c2d9bc03d031.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c2d9bc03d031.tar.gz` (if generated)